

D A Y 3

LOOPS & ITERATION

for • while • break • continue • pass • range()

for loop

while loop

Nested Loops

break/continue

range()

Production System



□ Day 3 — Complete Agenda

01

□ What is Iteration?

Why loops matter, real-world analogy

03

□ while Loop

Condition-based, infinite loops

05

□ Nested Loops

Loop inside loop, patterns

07

continue Statement

Skip current iteration

09

□ Counter & Accumulator

Loop patterns, sum, product

02

□ for Loop

Syntax, range(), lists, strings

04

□ range() Function

start, stop, step — all variants

06

break Statement

Exit loop early

08

□ pass Statement

Placeholder, do nothing

10

□ Industry Mini Project

Production Counter System

❏ What is Iteration?

Day 3

Iteration = repeating a block of code multiple times automatically. Instead of writing the same code 100 times, you write it ONCE inside a loop.

❏ Real World Loops

- ❏ Factory prints labels on 1000 bottles
- ❏ Alarm rings every hour
- ❏ Send email to all 500 students
- Do 3 sets of 10 push-ups
- ❏ Count from 1 to 1,000,000

❏ Without vs With Loop

```
❏ 100 lines:  
print(1)  
print(2)  
print(3)  
...  
print(100)
```

```
❏ 2 lines:  
for i in  
    range(1,101):  
    print(i)
```

❏ Python has 2 types of loops:

for loop — iterate over sequence

while loop — repeat while condition True

for Loop — Syntax & Basics

Day 3

The for loop iterates over a sequence (list, string, range, tuple...) one item at a time — automatically.

for_loop.py

```
# SYNTAX
for variable in sequence:
    # code block (indented)
    statement1
    statement2

# Example 1: counting
for i in range(1,6):
    print(f"Count: {i}")

# Example 2: list iteration
fruits = ["apple", "mango", "banana"]
for fruit in fruits:
    print(f"I like {fruit}")

# Example 3: string iteration
for char in "PYTHON":
    print(char, end=" ") # P Y T H O N
```

Output

```
Count:1
Count:2
Count:3
Count:4
Count:5
```

Key Points

- variable holds current item
- sequence can be any iterable
- body must be indented
- no {} needed — use indent
- loop variable usable outside

range() Function — All Variants

Day 3

`range(stop)`

`range(5)`

→ 0,1,2,3,4

Starts at 0, up to stop-1

`range(start,stop)`

`range(2,8)`

→ 2,3,4,5,6,7

start to stop-1

`range(start,stop,step)`

`range(0,10,2)`

→ 0,2,4,6,8

Even numbers

`range(stop,start,-step)`

`range(10,0,-2)`

→ 10,8,6,4,2

Countdown

for Loop — Common Patterns

Day 3

for_patterns.py

```
# Pattern 1: Sum of numbers
total = 0
for i in range(1, 101):
    total += i    # accumulator
print(f"Sum 1-100: {total}")    # 5050

# Pattern 2: Factorial
n = 5
fact = 1
for i in range(1, n+1):
    fact *= i
print(f"{n}! = {fact}")    # 5! = 120

# Pattern 3: Enumerate — index + value
names = ["Alice", "Bob", "Charlie"]
for index, name in enumerate(names):
    print(f"{index+1}. {name}")

# Pattern 4: zip — two lists together
scores = [85, 92, 78]
for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

Outputs

Sum of all

Sum 1-100: 5050

Factorial

5! = 120

enumerate

1. Alice
2. Bob
3. Charlie

zip

Alice: 85
Bob: 92
Charlie: 78

Iterating Strings & Lists

Day 3

string_list_loops.py

```
# — Strings —————
word = "PYTHON"

# Character by character
for ch in word:
    print(ch, end="-") # P-Y-T-H-O-N-

# Check vowels
vowels = "AEIOU"
for ch in word:
    if ch in vowels:
        print(f"{ch} is a vowel")

# Count characters
count = 0
for ch in "hello world":
    if ch != " ":
        count += 1
print(f"Letters: {count}") # 10

# — Lists —————
marks = [72, 88, 65, 91, 55, 78]

total = 0
for m in marks:
    total += m
avg = total / len(marks)
print(f"Average: {avg:.1f}") # Average: 74.8

high_scorers = []
for m in marks:
    if m >= 75:
```

? Output

```
P-Y-T-H-O-N-
H is a vowel
O is a vowel
Letters: 10
Average: 74.8
High: [88, 91, 78]
```

? Key Methods

- ▶ len(seq) — total items
- ▶ enumerate() — idx+val
- ▶ in — membership check
- ▶ append() — add to list
- ▶ range(len(seq)) — by index

□ while Loop — Condition-Based Repetition

Day 3

while loop keeps running AS LONG AS the condition is True. Use it when you don't know HOW MANY times to repeat.

□ while_loop.py

```
# Syntax
while condition:
    # body — runs repeatedly
    # Must eventually make condition False!

# Example 1: Count from 1 to 5
count = 1
while count <= 5:
    print(f"Count: {count}")
    count += 1    # ← update! or infinite loop

# Example 2: Sum until 0 entered
total = 0
while True:
    n = int(input("Enter number (0 to stop): "))
    if n == 0:
        break
    total += n
print(f"Total: {total}")

# Example 3: Guess the number
secret = 42
guess = 0
attempts = 0
while guess != secret:
```

? for vs while

for

Known count
list/range

while

Unknown count
condition

⚠ While Loop Rules

- ? Initialize variable BEFORE loop
- ? Update variable INSIDE loop
- ? Condition must become False
- ? Use break for safety exit
- ? while True: + break = common



Infinite Loops — Control & Use Cases

Day 3

An infinite loop runs FOREVER — either by mistake (bug!) or intentionally (server, menu, game loop). Always needs a break condition.

□ `infinite_loop.py`

```
# □ ACCIDENTAL infinite loop (BUG!)
count=1
while count<= 5:
    print(count)
    # Forgot: count+= 1 ← BUG! runs forever

# □ INTENTIONAL — Menu-driven program
while True:
    print("\n=== MENU ===")
    print("1.Add item ")
    print("2.View item s")
    print("3.Exit")
    choice = input("Choose:")
    if choice == "1":
        item = input("Item name:")
        print(f"Added: {item}")
    elif choice == "2":
        print("Showing item s...")
    elif choice == "3":
        print("Goodbye!")
        break # ← exits the infinite loop
    else:
        print("Invalid choice, try again")

# □ Server-style loop
in port t i n e
```

? How to Stop

break

Exit from inside loop body

flag variable

Set running=False

Ctrl+C

Emergency keyboard stop

Tim out

Set max iterations

? Rule: Every infinite loop needs a guaranteed

□ Nested Loops — Loop Inside a Loop

Day 3

A nested loop has one loop inside another. For each iteration of the OUTER loop, the INNER loop completes all its iterations.

□ nested_loops.py

```
# Example 1: Multiplication table
for i in range(1,4):    # outer: rows
    for j in range(1,4): # inner: columns
        print(f"{i}x{j}={i*j}",end=" ")
    print()             # new line after each row
```

```
# Example 2: Star pattern
rows = 5
for i in range(1, rows+1):
    for j in range(i):
        print("*",end="")
    print()
```

```
# Example 3: 2D data processing
classroom = [
    ["Alice",85,92,78],
    ["Bob", 72,88,90],
    ["Carol",95,70,85]
]
for student in classroom:
    name = student[0]
    scores = student[1:]
    avg = sum(scores)/len(scores)
    print(f'{name}: {avg:.1f}')

```

□ Output 1

```
1x1=1 1x2=2 1x3=3
2x1=2 2x2=4 2x3=6
3x1=3 3x2=6 3x3=9
```

□ Output 2 & 3

```
*
**
***
****
*****

Alice:85.0
Bob: 83.3
Carol:83.3
```

break Statement — Exit Loop Early

Day 3

break immediately EXITS the loop — no more iterations, no matter how many were remaining. Resumes after the loop.

□ break_exam_ples.py

```
# Exam ple 1: Find first even number
num bers = [3,7,9,4,11,2,8]
for n in num bers:
    if n % 2 == 0:
        print(f"First even: {n}") # 4
        break # stop looking!
print("Search done")

# Exam ple 2: Password attempts
m ax_tries = 3
for attempt in range(1, m ax_tries+1):
    p w d = input(f"Attempt {attempt}: ")
    if p w d == "secret":
        print(" Access granted!")
        break
    print(" Wrong password")
else: # for-else: runs if NO break
    print(" Account locked!")

# Exam ple 3: Production line — stop on defect
item s = ["ok","ok","ok","D E F E C T","ok","ok"]
produced = 0
for item in item s:
    if item == "D E F E C T":
        print(f" ^ Defect! Stopping line after {produced} item s")
```

? Flow with break

```
for item in list:
    ↓
    check condition
    ↓ True
    break —————→ EXIT
    ↓ False           ↓
    continue         after loop
iteration
    ↓
    next item
```

continue Statement — Skip an Iteration

Day 3

continue SKIPS the rest of the current iteration and jumps to the NEXT one. The loop itself doesn't stop.

□ continue_examples.py

```
# Example 1: Skip even numbers
for i in range(1, 11):
    if i % 2 == 0:
        continue # skip even, go to next i
    print(i, end=" ") # prints: 1 3 5 7 9

# Example 2: Skip empty inputs
names = ["Alice", "", "Bob", "", "Carol", ""]
for name in names:
    if not name.strip(): # empty or whitespace
        continue
    print(f"Hello, {name}!")

# Example 3: Process valid marks only
marks = [85, -5, 92, 110, 78, -1, 65]
valid_total = 0
valid_count = 0
for m in marks:
    if m < 0 or m > 100:
        print(f"△ Invalid mark: {m} — skipped")
        continue
    valid_total += m
    valid_count += 1
if valid_count > 0:
    print(f"Average: {valid_total / valid_count}")
```

? Output 1 & 2

```
1 3 5 7 9
Hello, Alice!
Hello, Bob!
Hello, Carol!
```

break vs continue

break

Exits loop completely

continue

Skips rest of current step

pass Statement — The Placeholder

Day 3

pass does absolutely NOTHING — it's a placeholder for code you haven't written yet. Keeps syntax valid with an empty block.

pass_exam.py

```
# pass keeps syntax valid — no error!

# 1. Empty loop body (future implementation)
for i in range(10):
    pass      # TODO: process items later

# 2. Empty if/else (skeleton)
age = 17
if age >= 18:
    pass      # TODO: grant access
else:
    pass      # TODO: deny access

# 3. Empty function / class placeholder
def process_data():
    pass      # implement later — no error!

class DataProcessor:
    pass      # empty class — valid!

# 4. Ignore specific error values
scores = [90, -1, 85, None, 78]
for i in scores:
    if i is None or i < 0:
        pass      # silently skip invalid
```

⚡ break vs continue vs pass

break

Exits loop entirely

continue

Skips to next iteration

pass

Does nothing
(placeholder)

```
# Quick comparison
for i in range(5):
    if i == 2:
```

□ for-else & while-else (Unique to Python!)

Day 3

Python's unique feature: else block after a loop. It runs ONLY if the loop completed WITHOUT hitting a break.

□ for_else.py

```
# for-else: search for a prime number
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            print(f"{n} divisible by {i}")
            break          # ← break if not prime
    else:
        return True       # ← else only if no break
    return False
```

```
print(is_prime(17)) # True
print(is_prime(12)) # False
```

```
# while-else: retry with limit
import random
target = random.randint(1,5)
attempts = 0
while attempts < 3:
    guess = int(input("Guess 1-5: "))
    attempts += 1
    if guess == target:
        print("✓ Correct!")
        break
```

? Rule

```
for .../while ...:
    if condition:
        break  # ← NO else
```

```
else:          # runs if
                # NO break hit
    # loop
    # completed
    # normally
```

□ Counter & Accumulator Patterns

Day 3

□ counter_accumulator.py

```
# — COUNTER PATTERN —————  
votes = ["Yes","No","Yes","No","Yes","No"]  
yes_count=0 # initialize counter  
no_count=0
```

```
for v in votes:  
    if v == "Yes":  
        yes_count+=1  
    else:  
        no_count+=1
```

```
print(f"Yes: {yes_count}, No: {no_count}")
```

— ACCUMULATOR PATTERN —————

```
# Sum  
prices = [29.99, 15.50, 8.75, 45.00]  
total=0.0  
for p in prices:  
    total+=p  
print(f"Cart total: ₹{total:2f}")
```

```
# Maximum / Minimum  
temps = [28, 35, 22, 40, 18, 33]  
max_t = temps[0] # start with first  
min_t = temps[0]  
for t in temps:  
    if t > max_t: max_t = t  
    if t < min_t: min_t = t  
print(f"Max: {max_t}°C, Min: {min_t}°C")
```

```
# String builder  
words = ["Python", "is", "awesome"]  
result = ""  
for w in words:
```

Counter Pattern

```
count=0  
for item in seq:  
    if condition:  
        count+=1
```

Accumulator Pattern

```
total=0  
for item in seq:  
    total+= item  
# total= sum
```



Input-Driven Loops

Day 3

□ input_loops.py

```
# Pattern 1: Sentinel-controlled loop
print("Enter marks (type -1 to stop):")
marks_list = []
while True:
    m = int(input("Mark: "))
    if m == -1: # sentinel value
        break
    if 0 <= m <= 100:
        marks_list.append(m)
    else:
        print("Invalid! Enter 0-100")

if marks_list:
    print(f"Count: {len(marks_list)}")
    print(f"Avg: {sum(marks_list)/len(marks_list):.1f}")
    print(f"Max: {max(marks_list)}")
```

```
# Pattern 2: Count-controlled input
n = int(input("How many students? "))
students = []
for i in range(1, n+1):
    name = input(f"Student {i} name: ")
    marks = float(input(f"Student {i} marks: "))
    students.append((name, marks))
```

```
# Sort and display
students.sort(key=lambda x: x[1], reverse=True)
print("\n---RANKING ---")
for rank, (name, m) in enumerate(students, 1):
    print(f"{rank}. {name}: {m}")
```

? Sentinel Pattern

A special value that signals 'stop input'

Common sentinels:

-1, 0, 'done',
'quit', empty str

? Count-Controlled

Ask how many first,
then loop exactly
that many times

```
n = int(input())
for i in range(n):
    ...
```


□ Menu-Driven Programs

Day 3

□ menu_program.py

```
# Student Management System - Menu Driven
students = {}
```

```
while True:
```

```
    print("\n" + "="*35)
    print(" STUDENT MANAGEMENT SYSTEM ")
    print("="*35)
    print(" 1. Add Student")
    print(" 2. View All Students")
    print(" 3. Search Student")
    print(" 4. Calculate Average")
    print(" 5. Exit")
    print("="*35)
```

```
    choice = input("Enter choice (1-5): ").strip()
```

```
    if choice == "1":
```

```
        name = input("Name: ")
        marks = float(input("Marks: "))
        students[name] = marks
        print(f" {name} added!")
```

```
    elif choice == "2":
```

```
        if not students:
            print("No students yet.")
        else:
            for name, marks in students.items():
                grade = "A" if marks >= 90 else "B" if marks >= 75 else "C"
                print(f" {name}: {marks} ({grade})")
```

```
    elif choice == "3":
```

```
        name = input("Search name: ")
        if name in students:
            print(f"Found: {name} = {students[name]}")
```

□ Loop Patterns — Stars & Numbers

Day 3

□ patterns.py

```
# Pattern 1: Right triangle
n = 5
for i in range(1, n+1):
    print("* " * i)

# Pattern 2: Inverted triangle
for i in range(n, 0, -1):
    print("* " * i)

# Pattern 3: Number pyramid
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j, end=" ")
    print()

# Pattern 4: Diamond
for i in range(1, n+1):
    print(" " * (n-i) + "*" * (2*i-1))
for i in range(n-1, 0, -1):
    print(" " * (n-i) + "*" * (2*i-1))
```

? Outputs

Right ▶

```
*
* *
* * *
* * * *
* * * * *
```

Inverted ▼

```
* * * * *
* * * *
* * *
* *
*
```

Numbers

```
1
1 2
1 2 3
1 2 3 4
```

□ Loop Debugging Techniques

Day 3

□ debug_bops.py

```
# Technique 1: Print inside the loop (trace)
total = 0
for i in range(1, 6):
    total += i
    print(f" DEBUG : i={i}, total={total}") # trace
print(f"Final: {total}")
```

```
# Technique 2: Add iteration counter
count = 0
while condition:
    count += 1
    if count > 1000: # safety guard!
        print("⚠ Possible infinite loop!")
        break
    # ... loop body
```

```
# Technique 3: Dry run (trace by hand)
# num s = [1, 2, 3, 4, 5]
# Iteration | i | result | condition
# 1 | 1 | 1 | i < 5 True
# 2 | 2 | 3 | i < 5 True
# 3 | 3 | 6 | i < 5 True
# 4 | 4 | 10 | i < 5 True
# 5 | 5 | 15 | i < 5 False ← stops
```

```
# Technique 4: Check off-by-one errors
for i in range(5): # 0, 1, 2, 3, 4 → 5 items
    pass
for i in range(1, 6): # 1, 2, 3, 4, 5 → 5 items too
    pass
```

❓ Common Loop Bugs

Off-by-one

range(5) gives 0-4
Not 1-5

Infinite loop

Forgot to update
loop variable

Wrong indent

Code outside loop
runs only once

Wrong operator

= instead of ==
in condition

Modifying list

Don't change list
while iterating

Time Delay Simulation Using Loops

Day 3

```
□ time_simulation.py
```

```
import time

# Simulation 1: Loading bar
print("Loading:", end="", flush=True)
for i in range(20):
    print("█", end="", flush=True)
    time.sleep(0.1)    # 0.1 second delay
print(" Done! □")

# Simulation 2: Countdown timer
print("\n□ Countdown:")
for i in range(10, 0, -1):
    print(f" {i}...", end="\r") # overwrite line
    time.sleep(1)
print(" LAUNCH! □ ")

# Simulation 3: Machine cycle simulation
machine_speed = 0.5 # seconds per item
items_to_produce = 10

print(f"\n□ Starting production ({items_to_produce} items):")
for cycle in range(1, items_to_produce+1):
    time.sleep(machine_speed)
    progress = "█" * cycle + "░" * (items_to_produce - cycle)
    pct = cycle / items_to_produce * 100
    print(f" [{progress}] {pct:0f}% - Item {cycle} done", end="\r")

print(f"\n□ Production complete!")

# Simulation 4: Heartbeat monitor
for beat in range(5):
    print("□ BEAT", beat+1)
```

□ Advanced: List Comprehensions

Day 3

List comprehension = compact way to create lists using a for loop in ONE line. Very Pythonic and fast!

□ com prehensions.py

Syntax: [expression for item in iterable if condition]

Normal loop → list

```
squares = []  
for x in range(1,6):  
    squares.append(x**2)    # [1,4,9,16,25]
```

List comprehension → same result, 1 line

```
squares = [x**2 for x in range(1,6)]
```

With condition

```
evens = [x for x in range(20) if x % 2 == 0]  
# [0,2,4,6,8,10,12,14,16,18]
```

Transform strings

```
names = ["alice", "bob", "carol"]  
upper = [name.upper() for name in names]  
# ["ALICE", "BOB", "CAROL"]
```

Filter valid marks

```
marks = [85, -5, 92, 110, 78, 65]  
valid = [m for m in marks if 0 <= m <= 100]  
# [85, 92, 78, 65]
```

Nested comprehension (matrix)

□ while Loop — Advanced Patterns

Day 3

□ while_advanced.py

```
# Pattern 1: do while simulation in Python
while True:
    value = int(input("Enter positive number: "))
    if value > 0:
        break
    print("Must be positive!")
```

```
# Pattern 2: Fibonacci with while
a, b = 0, 1
lim = 100
print("Fibonacci up to", lim, ":")
while a <= lim:
    print(a, end=" ")
    a, b = b, a + b # swap trick!
# 0 1 1 2 3 5 8 13 21 34 55 89
```

```
# Pattern 3: Collatz Conjecture
n = int(input("Enter a number: "))
steps = 0
while n != 1:
    if n % 2 == 0:
        n = n // 2
    else:
        n = 3 * n + 1
    steps += 1
print(f"Reached 1 in {steps} steps")
```

```
# Pattern 4: ATM simulation
balance = 10000
while True:
    print(f"\nBalance: ₹{balance}")
    amt = float(input("Withdrawal (0=exit): "))
    if amt == 0: break
    if amt > balance: print("Insufficient!")
```

Loops with Strings — Deep Dive

Day 3

□ string_bops.py

```
# String analysis using loops
text= "Hello World Python 2024"
```

```
# Count each character type
letters = digits = spaces = others = 0
for ch in text:
    if ch.isalpha(): letters += 1
    elif ch.isdigit(): digits += 1
    elif ch == " ": spaces += 1
    else: others += 1
```

```
print(f"Letters:{letters} Digits:{digits} Spaces:{spaces}")
```

```
# Reverse a string
original= "PYTHON "
reversed_str= ""
for ch in original:
    reversed_str= ch + reversed_str # prepend
print(reversed_str)      # NOHTYP
```

```
# Caesar cipher— simple encryption
message= "HELLO "
shift = 3
cipher = ""
for ch in message:
    code = ord(ch)
    shifted = (code -65 + shift)% 26 + 65
    cipher+= chr(shifted)
print(f"Encrypted:{cipher}") # KHOOR
```

```
# Palindrome check
word = "RACECAR"
is_palindrome = True
for i in range(len(word)//2):
```

□ Industry Mini Project — Design Plan

Day 3

Production Counter System — Real factory simulation using ALL loop concepts: for, while, break, continue, pass, counters

1

Production Counter

Count items in each machine cycle

2

Shift Tracking

Morning/Evening/Night shift breakdown

3

Defect Detection

Flag & skip defective items (continue)

4

Auto-Stop at Target

Stop when target reached (break)

5

Downtime Simulation

Pause machine with time delay

6

Efficiency Report

Good items / Total items * 100

7

Worker Productivity

Items per worker per shift

8

Dashboard Display

Console-based production summary

Concepts used: for loop, while loop, break, continue, counters, accumulators, time.sleep, nested loops

□ Production Counter System — Code (Part 1)

Day 3

□ production_part1.py

```
import time, random

# == PRODUCTION COUNTER SYSTEM ==
TARGET = 50      # production target
SHIFTS = ["Morning", "Evening", "Night"]
WORKERS_PER_SHIFT = 5
DEFECT_RATE = 0.10 # 10% chance defective

total_produced = 0
total_defects = 0
shift_data = {}

print("=" * 45)
print("  PRODUCTION COUNTER SYSTEM ")
print("=" * 45)

for shift in SHIFTS:
    shift_produced = 0
    shift_defects = 0

    print(f"\n□ {shift} Shift Starting ...")
    print("-" * 30)

    for cycle in range(1, 21): # 20 cycles per shift

        if total_produced >= TARGET: # target reached
            print(f"\n□ TARGET {TARGET} REACHED ! Stopping.")
            break

        # Simulate machine cycle
        time.sleep(0.05) # fast simulation

        # Random defect check
        is_defect = random.random() < DEFECT_RATE
```

□ Production Counter System — Code (Part 2)

Day 3

□ production_part2.py

```
# Worker productivity per shift
productivity = shift_produced / WORKERS_PER_SHIFT
print(f"\n □ {shift} Shift Summary:")
print(f"   Items produced : {shift_produced}")
print(f"   Defects found   : {shift_defects}")
print(f"   Productivity    : {productivity:1f} items/worker")

# == FINAL DASHBOARD ==
total_cycles = total_produced + total_defects
efficiency = (total_produced / total_cycles * 100) if total_cycles > 0 else 0

print("\n" + "=" * 45)
print("   PRODUCTION FINAL REPORT")
print("=" * 45)
print(f" □ Target           : {TARGET}")
print(f" □ Total Produced   : {total_produced}")
print(f" □ Total Defects    : {total_defects}")
print(f" □ Efficiency       : {efficiency:1f}% ")
print("=" * 45)
print(" SHIFTS BREAKDOWN ")
print("-" * 45)
for shift, data in shift_data.items():
    bar = '█' * data['produced'] + '░' * (20 - data['produced'])
    print(f" {shift}<8}: [{bar}] {data['produced']}>2")
print("=" * 45)

if efficiency >= 90:
    print(" □ EXCELLENT PRODUCTION QUALITY!")
elif efficiency >= 75:
    print(" □ GOOD — Minor improvements needed")
else:
    print(" ^ POOR — Quality review required")
```

PRODUCTION COUNTER SYSTEM — Sample Output

Day 3

□ Morning Shift Starting...

□ Cycle 01:Item 001 OK

□ Cycle 02:Item 002 OK

△ Cycle 03:DEFECT detected — skipped

□ Cycle 04:Item 003 OK

□ Cycle 05:Item 004 OK

...

□ Morning Shift Summary:

Items produced :18

Defects found :2

Productivity :3.6 items/worker

□ TARGET 50 REACHED !Stopping.

PRODUCTION FINAL REPORT

□ Target :50

□ TotalProduced :50

△ TotalDefects :7

□ Efficiency :87.7%

SHIFTWISE BREAKDOWN

Morning:  118

Concepts Map — Production System

Day 3



for loop

Iterate shifts & cycles

```
for shift in SHIFTS
```



while loop

Could wrap entire system

```
while not done:
```

break

Stop at target

```
if total >= TARGET: break
```

continue

Skip defective items

```
if defect: continue
```



Counter

Count produced & defects

```
total_produced += 1
```



Accumulator

Track shift totals

```
shift_produced += 1
```



Nested loop

Shifts × Cycles

```
for shift: for cycle:
```

time.sleep()

Machine cycle delay

```
time.sleep(0.05)
```



random

Simulate defects

```
random.random() < 0.1
```



dict tracking

Per-shift data store

```
shift_data[shift] = {...}
```

Interview Questions — Day 3

Day 3

1

What is the difference between break and continue?

→ break exits the loop entirely; continue skips the current iteration and moves to the next one.

2

When would you use a while loop vs a for loop?

→ for loop when you know the number of iterations; while loop when you repeat until a condition changes.

3

What does the else clause do after a for loop?

→ It runs only if the loop completed without hitting a break — useful for search algorithms.

4

What is range(2, 10, 3)?

→ Generates: 2, 5, 8 — starts at 2, increments by 3, stops before 10.

5

How do you avoid an infinite loop?

→ Ensure the loop variable is updated, the condition will eventually become False, or use break with a counter guard.

6

What is a list comprehension?

→ A compact one-line syntax to create lists: [expr for item in iterable if condition].

□ Day 3 — Complete Cheat Sheet

Day 3

□ cheatsheet.py

——— F O R L O O P ———

```
for var in sequence: body      # basic
for i in range(5): body        # 0,1,2,3,4
for i in range(1,6): body      # 1,2,3,4,5
for i in range(0,10,2): body   # 0,2,4,6,8
for i,v in enumerate(lst): body # index + value
for a,b in zip(l1,l2): body    # two lists
```

——— W H I L E L O O P ———

```
while condition: body          # basic
while True: ... if done: break  # infinite + break
```

——— C O N T R O L S T A T E M E N T S ———

```
break          # exit loop entirely
continue       # skip to next iteration
pass           # do nothing (placeholder)
```

——— L O O P W I T H E L S E ———

```
for/while ...:
    if cond: break
else:          # runs only if NO break
```

——— P A T T E R N S ———

```
# Counter:   count = 0; for ...: count += 1
# Accumulator: total = 0; for ...: total += val
# Maximum:   mx = lst[0]; for v in lst: if v > mx: mx = v
```

——— L I S T C O M P R E H E N S I O N ———

```
result = [expr for item in seq if condition]
evens = [x for x in range(20) if x % 2 == 0]
```

Quick Practice Challenges

Day 3

1

? Number Pyramid

Print a number pyramid for n=5 using nested loops: row i prints i numbers (1 to i). Use for loops.

Easy

2

? Prime Numbers

Print all prime numbers from 2 to 100. A prime is divisible only by 1 and itself. Use for-else.

Medium

3

? ATM Machine

Simulate ATM: user has ₹10,000. Loop until balance=0 or user exits. Use while + break. Validate input.

Medium

4

? Word Frequency

Given a sentence, count how many times each word appears. Output sorted by frequency.

Hard

□ Assignment Tasks — Day 3

1 Task 1 — Number Pattern Generator (Intermediate)

Build a Python program that generates the following patterns based on user input n:

1. Right triangle of stars (nested for loops, range)
2. Inverted triangle of numbers (nested for, range with step -1)
3. Pascal's triangle first n rows (nested loops + accumulator)
4. Prime numbers up to n (for-else pattern, break inside inner loop)

2 Task 2 — Production Counter System (Industry Project)

Develop a full Production Counter System with ALL of the following features:

1. Input: target units, workers per shift, defect rate as a percentage
2. Simulate 3 shifts × 20 machine cycles using nested for loops
3. Randomly mark items as defective — skip them using continue
4. Stop ALL production when total target is reached using break
5. Track per-shift: items produced, defects, worker productivity

Day 3 — LOOPS MASTERED! 🏆

What you mastered today:

- ❓ for loop — sequences, range(), enumerate(), zip()
 - ❓ while loop — condition-based, input-driven
 - ❓ Infinite loops — controlled with break
 - ❓ Nested loops — patterns, 2D data
 - ❓ break, continue, pass — control flow
 - ❓ Counter & Accumulator patterns
 - ❓ Loop with strings, lists, menu programs
 - ❓ Industry Project: Production Counter System
- Complete your assignments! 📝**

